

MCQ 1 —

What is the output?

```
public class Test {  
  
    static int method() {  
  
        try {  
            System.out.print("A ");  
  
            try {  
                System.out.print("B ");  
                throw new ArithmeticException();  
            }  
            catch (NullPointerException e) {  
                System.out.print("C ");  
            }  
            finally {  
                System.out.print("D ");  
            }  
  
            return 10;  
        }  
        catch (ArithmeticException e) {  
            System.out.print("E ");  
            return 20;  
        }  
        finally {  
            System.out.print("F ");  
        }  
    }  
  
    public static void main(String[] args) {  
        System.out.print(method());  
    }  
}
```

Options

A.

A B C D F 10

B.

A B D E F 20

C.

A B D E 20

D.

A B E D F 20

MCQ 2 —

What is the output?

```
public class Test {  
  
    static int calculate() {  
  
        try {  
            System.out.print("A ");  
            throw new RuntimeException();  
        }  
        catch (RuntimeException e) {  
            System.out.print("B ");  
            throw new IllegalArgumentException();  
        }  
        finally {  
            System.out.print("C ");  
            return 100;  
        }  
    }  
  
    public static void main(String[] args) {  
  
        try {  
            System.out.print(calculate());  
        }  
        catch (Exception e) {  
            System.out.print("D ");  
        }  
  
        System.out.print("E");  
    }  
}
```

Options

A.

A B C D E

B.

A B C 100 E

C.

A C 100 E

D.

A B D E

MCQ 3 —

What is the output?

```
public class Test {  
  
    static int method() {  
  
        int x = 10;  
  
        try {  
            System.out.print("A ");  
            return x++;  
        }  
        finally {  
            System.out.print("B ");  
            x = 50;  
            System.out.print(x + " ");  
        }  
    }  
  
    public static void main(String[] args) {  
  
        System.out.print(method());  
    }  
}
```

Options

A.

A B 50 10

B.

A B 50 11

C.

A 10 B 50

D.

A B 11 50

MCQ 4 —

What is the output?

```
public class Test {  
  
    static int method() {  
  
        int x = 10;  
  
        try {  
            System.out.print("A ");  
            return x++;  
        }  
        finally {  
            System.out.print("B ");  
            x = 50;  
            System.out.print(x + " ");  
        }  
    }  
  
    public static void main(String[] args) {  
  
        System.out.print(method());  
    }  
}
```

Options

A.

A B 50 10

B.

A B 50 11

C.

A 10 B 50

D.

A B 11 50

MCQ 5 —

What is the output?

```
class A {  
  
    int x = 10;  
  
    A() {  
        System.out.print("A1-" + x + " ");  
    }  
}
```

```

        display();
        System.out.print("A2-" + x + " ");
    }

    void display() {
        System.out.print("AD-" + x + " ");
    }
}

class B extends A {

    int x = 20;

    B() {
        System.out.print("B1-" + x + " ");
        display();
        System.out.print("B2-" + super.x + " ");
    }

    @Override
    void display() {
        System.out.print("BD-" + x + "-" + super.x + " ");
    }
}

class C extends B {

    int x = 30;

    C() {
        System.out.print("C1-" + x + " ");
        super.display();
        System.out.print("C2-" + super.x + " ");
    }

    @Override
    void display() {
        System.out.print("CD-" + x + "-" + super.x + " ");
    }
}

public class Test {

    public static void main(String[] args) {

        A obj = new C();

        System.out.print("| ");

        obj.display();

        System.out.print(" | " + obj.x);
    }
}

```

Options

A.

A1-10 CD-0-0 A2-10 B1-20 CD-0-20 B2-10 C1-30 BD-20-10 C2-20 | CD-30-20 | 10

B.

A1-10 AD-10 A2-10 B1-20 BD-20-10 B2-10 C1-30 CD-30-20 C2-20 | CD-30-20 | 10

C.

A1-10 CD-0-0 A2-10 B1-20 CD-0-20 B2-10 C1-30 BD-20-10 C2-20 | CD-30-20 | 30

D.

A1-10 CD-30-20 A2-10 B1-20 CD-30-20 B2-10 C1-30 CD-30-20 C2-20 | CD-30-20 | 10

MCQ 6 —

```
class Parent {  
  
    void process(Object obj) {  
        System.out.print("PO ");  
    }  
  
    void process(String str) {  
        System.out.print("PS ");  
    }  
  
    static void display(Object obj) {  
        System.out.print("PDO ");  
    }  
}
```

```
class Child extends Parent {  
  
    @Override  
    void process(Object obj) {  
        System.out.print("CO ");  
    }  
  
    void process(Integer i) {  
        System.out.print("CI ");  
    }  
  
    static void display(Object obj) {  
        System.out.print("CDO ");  
    }  
  
    static void display(String str) {  
        System.out.print("CDS ");  
    }  
}
```

```
public class Test {
```

```

public static void main(String[] args) {

    Parent p = new Child();
    Child c = new Child();

    p.process(null);
    c.process(null);

    Object obj = null;

    p.process(obj);
    c.process(obj);

    p.display(null);
    c.display(null);
}
}

```

Options

A.

PS PS CO CO PDO CDS

B.

PS PS CO CO PDO CDO

C.

CO PS CO CO PDO CDS

D.

Compilation Error

MCQ 7 —

What is the output?

```
import java.util.*;
```

```
public class Test {
```

```
    public static void main(String[] args) {
```

```
        LinkedList<Integer> list = new LinkedList<>();
```

```
        list.offer(10);
```

```
        list.offerFirst(20);
```

```
        list.offerLast(30);
```

```
        list.push(40);
```

```
        int a = list.pop();
```

```
        list.addFirst(a + list.pollLast());
```

```

list.offer(list.removeFirst() * 2);

list.add(list.peekFirst() + list.peekLast());

System.out.println(list);

System.out.println(
    list.pollFirst()
    + list.pollLast()
    + list.peek()
);
}
}

```

Options

A.

[20, 10, 60, 80]
110

B.

[20, 10, 140]
170

C.

[70, 20, 10, 140]
220

D.

[70, 10, 140]
220

MCQ 8 —

What is the output?

```

interface Printable {

    void print();

    default void message() {
        System.out.print("Printable ");
    }
}

class Parent {

    void display() {
        System.out.print("Parent-Display ");
    }

    void common() {
        System.out.print("Parent-Common ");
    }
}

```

```
}  
}
```

```
class Child extends Parent implements Printable {
```

```
    @Override  
    public void print() {  
        System.out.print("Child-Print ");  
    }  
  
    @Override  
    void display() {  
        System.out.print("Child-Display ");  
    }  
  
    @Override  
    public void message() {  
        System.out.print("Child-Message ");  
    }  
  
    void childMethod() {  
        System.out.print("Child-Method ");  
    }  
}
```

```
public class Test {
```

```
    public static void main(String[] args) {  
  
        Parent p = new Child();  
        Printable i = new Child();  
        Child c = new Child();  
  
        p.display();  
        p.common();  
  
        System.out.print(" | ");  
  
        i.print();  
        i.message();  
  
        System.out.print(" | ");  
  
        c.display();  
        c.common();  
        c.print();  
        c.message();  
        c.childMethod();  
  
        System.out.print(" | ");  
  
        ((Printable) p).print();  
        ((Child) p).childMethod();  
    }
```

}

Options

A

Child-Display Parent-Common |
Child-Print Child-Message |
Child-Display Parent-Common Child-Print Child-Message Child-Method |
Child-Print Child-Method

B

Parent-Display Parent-Common |
Child-Print Printable |
Child-Display Parent-Common Child-Print Printable Child-Method |
Child-Print Child-Method

C

Child-Display Parent-Common |
Child-Print Printable |
Child-Display Parent-Common Child-Print Child-Message Child-Method |
Printable Child-Method

D

Compilation Error

MCQ 9 —

What is the output of the following program?

```
class Animal {  
  
    void sound() {  
        System.out.print("Animal-Sound ");  
    }  
  
    void eat() {  
        System.out.print("Animal-Eat ");  
    }  
}  
  
class Dog extends Animal {  
  
    @Override  
    void sound() {  
        System.out.print("Dog-Bark ");  
    }  
  
    @Override  
    void eat() {  
        System.out.print("Dog-Eat ");  
    }  
  
    void guard() {  
        System.out.print("Dog-Guard ");  
    }  
}
```

```

class Puppy extends Dog {

    @Override
    void sound() {
        System.out.print("Puppy-Bark ");
    }

    @Override
    void eat() {
        System.out.print("Puppy-Eat ");
    }

    void play() {
        System.out.print("Puppy-Play ");
    }
}

public class Test {

    public static void main(String[] args) {

        Animal a1 = new Dog();
        Animal a2 = new Puppy();
        Dog d1 = new Puppy();
        Puppy p1 = new Puppy();

        a1.sound();
        a1.eat();

        System.out.print("| ");

        a2.sound();
        a2.eat();

        System.out.print("| ");

        d1.sound();
        d1.eat();
        d1.guard();

        System.out.print("| ");

        p1.sound();
        p1.eat();
        p1.guard();
        p1.play();

        System.out.print("| ");

        ((Dog) a2).guard();
        ((Puppy) a2).play();
    }
}

```

Options

A

Dog-Bark Dog-Eat |
Puppy-Bark Puppy-Eat |
Puppy-Bark Puppy-Eat Dog-Guard |
Puppy-Bark Puppy-Eat Dog-Guard Puppy-Play |
Dog-Guard Puppy-Play

B

Animal-Sound Animal-Eat |
Animal-Sound Animal-Eat |
Dog-Bark Dog-Eat Dog-Guard |
Puppy-Bark Puppy-Eat Dog-Guard Puppy-Play |
Dog-Guard Puppy-Play

C

Dog-Bark Dog-Eat |
Puppy-Bark Puppy-Eat |
Dog-Bark Dog-Eat Dog-Guard |
Puppy-Bark Puppy-Eat Dog-Guard Puppy-Play |
Dog-Guard Puppy-Play

D

Dog-Bark Dog-Eat |
Puppy-Bark Puppy-Eat |
Puppy-Bark Puppy-Eat Dog-Guard |
Puppy-Bark Puppy-Eat Dog-Guard Puppy-Play |
ClassCastException

MCQ 10 —

An order service performs:

POST /orders



Save Order in Database



Send OrderCreated event to Kafka

Code:

@Transactional

```
public void createOrder(Order order) {
```

```
    orderRepository.save(order);
```

```
    kafkaTemplate.send(  
        "order-created",  
        order.getId().toString(),  
        new OrderCreated(order.getId())  
    );
```

```
}
```

Scenario:

1. Database transaction commits successfully.
2. Application crashes immediately afterward.
3. Kafka event was never successfully delivered.

The database now contains the order, but the payment service never receives an event.

What is the BEST architecture to solve this problem?

A.

Call Kafka before saving the order.

B.

Retry the REST request from the frontend.

C.

Use the Transactional Outbox Pattern.

D.

Make the Kafka consumer poll the order database directly.

MCQ 11 —

Payment service consumes:
order-created

Consumer:

```
@KafkaListener(  
    topics = "order-created",  
    groupId = "payment-group"  
)  
public void process(OrderCreated event) {  
  
    paymentService.createPayment(  
        event.orderId()  
    );  
}
```

Scenario:

1. Consumer receives OrderCreated(orderId=101).
2. Payment is successfully created.
3. Consumer crashes before Kafka offset is committed.
4. Consumer restarts.
5. Kafka sends orderId=101 again.

Now two payments may be created.

What is the BEST solution?

A.

Set the Kafka topic to only one partition.

B.

Use a different consumer group.

C.

Make the consumer idempotent using a unique event/order identifier.

D.

Disable retries in Kafka.

MCQ 12 —

```
@RestController
@RequestMapping("/users")
class UserController {

    @GetMapping("/{id}")
    String getUser(@PathVariable Long userId) {
        return "User " + userId;
    }
}
```

Request:
GET /users/10

What happens?

- A. User 10
- B. User null
- C. 400/500 error due to missing path variable name
- D. 404 Not Found

MCQ 13 —

```
@GetMapping("/search")
String search(@RequestParam String name) {
    return name;
}
```

Request:
GET /search

What happens?

- A. null
- B. Empty String
- C. 400 Bad Request
- D. 404 Not Found

MCQ 14 —

```
@RestController
class TestController {

    @GetMapping("/data")
    String getData() {
        return "A";
    }

    @GetMapping("/data")
    Integer getNumber() {
        return 100;
    }
}
```

```
}  
}
```

What happens?

- A. A
- B. 100
- C. Both methods execute
- D. Application fails because of ambiguous mapping

MCQ 15 —

```
@PostMapping("/users")  
String create(@RequestBody User user) {  
    return user.getName() + "-" + user.getAge();  
}
```

Request body:

```
{  
    "name": "Viren"  
}
```

age is: private int age;

What is returned?

- A. Viren-null
- B. Viren-0
- C. 400 Bad Request
- D. Compilation Error

MCQ 16 —

```
@RestController  
@RequestMapping("/api")  
class TestController {  
  
    @GetMapping("/users")  
    String users() {  
        return "Users";  
    }  
}
```

Request:

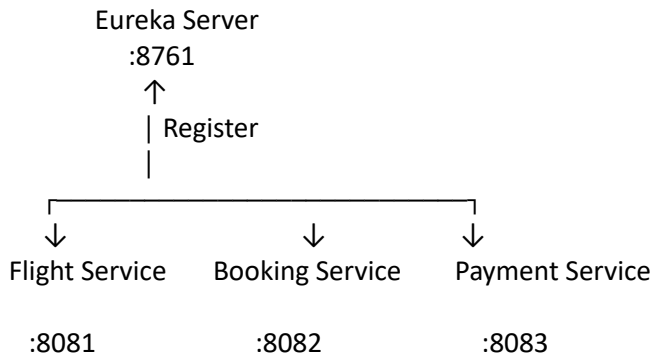
GET /users

What happens?

- A. Users
- B. 404 Not Found
- C. 400 Bad Request
- D. Compilation Error

MCQ 17—

A company has the following architecture:

**Flight Service configuration:**

```
spring.application.name=FLIGHT-SERVICE
```

```
server.port=8081
```

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Booking Service uses:

```
spring.application.name=BOOKING-SERVICE
```

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

However, the developer accidentally changes the Flight Service application name to:

```
spring.application.name=fight-service
```

The Booking Service Feign client is:

```
@FeignClient(name = "FLIGHT-SERVICE")
```

```
public interface FlightClient {
```

```
    @GetMapping("/api/flights/{flightNumber}")
    String getFlight(@PathVariable String flightNumber);
}
```

What is most likely to happen?

- A. Feign automatically converts the service name and successfully calls the service.
- B. Booking Service gets 404 Not Found.
- C. Eureka registers the service under a name that does not match the Feign client's lookup name, so service discovery may fail.
- D. The Flight Service will not start.

MCQ 18 —

A Booking Service calls Flight Service through Feign.

```

@CircuitBreaker(
    name = "flightService",
    fallbackMethod = "flightFallback"
)
public String bookFlight(String flightNumber) {

    return flightClient.getFlight(flightNumber);
}

public String flightFallback(
    String flightNumber,
```

```

Exception ex) {

    return "Flight service unavailable for "
        + flightNumber;
}

```

Resilience4j configuration:

```

resilience4j.circuitbreaker.instances.flightService.failure-rate-threshold=50
resilience4j.circuitbreaker.instances.flightService.sliding-window-size=4
resilience4j.circuitbreaker.instances.flightService.minimum-number-of-calls=4

```

The Flight Service is completely down.
The Booking API receives 4 requests.

What happens after the first 4 failed calls?

- A. Circuit breaker opens immediately after the first failure.
- B. All 4 calls fail and use the fallback; after the failure threshold is evaluated, the circuit can move to OPEN.
- C. Eureka automatically starts another Flight Service.
- D. Feign retries forever.

MCQ 19 —

- An **Order Service** and **Payment Service** are registered with **Eureka Server**.
- Order Service uses a **Feign Client** to call Payment Service.
- Payment Service crashes.
- The first few requests fail and the **Circuit Breaker becomes OPEN**.
- Payment Service is restarted and successfully registers again with Eureka.
- Immediately after registration, a new request reaches Order Service.
- The Circuit Breaker's configured waiting time has **not yet expired**.

What happens?

- A. Feign immediately calls the restarted Payment Service because Eureka shows it as available.
- B. Eureka closes the Circuit Breaker automatically.
- C. Circuit Breaker blocks the Feign call and the fallback executes.
- D. API Gateway directly calls Payment Service.

MCQ 20 —

- Two instances of **Product Service** are running:
 PRODUCT-SERVICE-1 → Port 8081
 PRODUCT-SERVICE-2 → Port 8082
- Both instances are registered in **Eureka** with the same service name:
 PRODUCT-SERVICE
- API Gateway route uses:
 lb://PRODUCT-SERVICE
- A client sends 10 requests through API Gateway.
- After the first 5 requests, PRODUCT-SERVICE-1 crashes.
- Eureka has not yet removed the crashed instance from its registry.

What can happen to the next requests?

- A. All requests immediately go only to PRODUCT-SERVICE-2.
- B. Some requests may still be routed toward the stale/crashed instance until service discovery updates.
- C. API Gateway automatically restarts PRODUCT-SERVICE-1.
- D. The API Gateway stops permanently because one service instance crashed.